

NEXUS PROTOCOL V2.0



Nexus Messenger

 $A + F = C . B$

Yeni nesil iletişim protokolü ile kesintisiz ve şık bir deneyim. Gerçek zamanlı mesajlaşma, sesli/görüntülü arama, yönetici paneli ve daha fazlası.

Kapsamlı Sistem Sunumu — 12 Haziran 2026

1. Proje Genel Bakış

Nexus Messenger (kod adı: **A+F=C.B**), Firebase altyapısı üzerine kurulmuş, gerçek zamanlı, çok platformlu bir mesajlaşma ve iletişim uygulamasıdır. Web, mobil (Android/Capacitor) ve GitHub Pages üzerinden erişilebilir. Proje, modern bir teknoloji yığını kullanarak WhatsApp benzeri bir kullanıcı deneyimini hedeflerken; yönetici paneli, broadcast sistemi, şifreli mesajlaşma ve WebRTC tabanlı sesli/görüntülü arama gibi gelişmiş özellikler sunar.

ÖZELLİK	DETAY
Canlı Site	aydannadya31.github.io/nexus-messenger/
Proje Adı	Nexus Messenger / A+F=C.B
Platformlar	Web (PWA) + Android (Capacitor)
Mimari	SPA (Single Page Application) + Firebase Backend
Dağıtım	GitHub Pages (otomatik CI/CD) & Firebase Deploy
Firebase Projesi	gen-lang-client-0308378658
Firestore Database	ai-studio-6f70c272-6822-4c0b-a15d-b77d18f46fb0 (named database)

Temel Özellikler

Gerçek Zamanlı Mesajlaşma

Anlık metin, resim, video ve ses mesajları. Emoji tepkileri, okundu bilgisi, şifreli mesajlar.

Sesli & Görüntülü Arama

WebRTC tabanlı peer-to-peer arama. STUN/TURN desteği, grup arama, zil sesi.

Yönetici Paneli

Kullanıcı yönetimi, ban sistemi, mesaj denetleme, etik kuralları, kalıcı silme.

Broadcast Sistemi

Ülke bazlı küresel mesajlaşma. Günlük limit ve cooldown kontrollü.

Grup Sohbetleri

Şifre korumalı gruplar, ülke etiketi, yönetici onaylı katılım.

Güvenlik

Firestore kuralları, şifreli mesajlar, ban sistemi, 12 güvenlik senaryosu.

2. Teknoloji Yığını (Tech Stack)

2.1 Frontend

TEKNOLOJİ	VERSİYON	KULLANIM AMACI
React	^19.0.1	Kullanıcı arayüzü bileşenleri
TypeScript	~5.8.2	Tip güvenliği ve derleme
Vite	^6.2.3	Build aracı ve geliştirme sunucusu
Tailwind CSS	^4.1.14	Utility-first CSS framework
Motion (Framer Motion)	^12.23.24	Animasyonlar ve geçiş efektleri
Lucide React	^0.546.0	SVG ikon seti
clsx + tailwind-merge	^2.1.1 / ^3.6.0	Dinamik CSS sınıfı yönetimi
date-fns	^4.1.0	Tarih/saat biçimlendirme
react-markdown	^10.1.0	Markdown işleme

2.2 Backend & Altyapı

TEKNOLOJİ	KULLANIM AMACI
Firebase Authentication	Google Sign-In ile kimlik doğrulama
Cloud Firestore	Gerçek zamanlı NoSQL veritabanı
Firestore Security Rules	Veri güvenliği ve yetkilendirme
GitHub Actions	CI/CD - otomatik build ve deploy
GitHub Pages	Web barındırma (static site)
Capacitor 8	Android native paketlenme (WebView)
WebRTC	Peer-to-peer sesli/görüntülü arama
Google Gemini API	AI entegrasyonu (yapılandırılmış)

2.3 Geliştirme Araçları

- **TypeScript Compiler** (tsc --noEmit) - Statik tip kontrolü
- **tsx** - TypeScript script çalıştırma
- **dotenv** - Çevre değişkeni yönetimi
- **Express** (^4.21.2) - Geliştirme sunucusu (functions)

- **Firebase CLI** - Firestore kuralları ve indeks deploy

3. Sistem Mimarisi

3.1 Bileşen Hiyerarşisi

```
main.tsx
├─ ErrorBoundary // Hata yakalama katmanı
├─ App.tsx
│   ├─ AuthProvider // Kimlik doğrulama + Firestore profil senkronizasyonu
│   │   └─ CallProvider // WebRTC çağrı state yönetimi
│   │       └─ NexusApp // Ana uygulama gövdesi
│   │           ├─ Login
│   │           ├─ Sidebar
│   │           ├─ ChatArea
│   │           ├─ BroadcastChannel
│   │           ├─ NewChatModal / BroadcastModal
│   │           ├─ CallOverlay
│   │           └─ ProfileSetup / ProfileModal
│   └─ Firebase Services
│       ├─ Authentication (Google Sign-In)
│       └─ Firestore (onSnapshot real-time listeners)
```

3.2 Veri Akışı ve Gerçek Zamanlı Model

Tüm veri akışı **Firestore gerçek zamanlı dinleyicileri** (onSnapshot) üzerinden sağlanır. Geleneksel REST API yerine Firebase'in real-time listener modeli kullanılır. Bu sayede:

DİNLEYİCİ	TETİKLEYİCİ	KULLANILDIĞI BİLEŞEN
onAuthStateChanged	Giriş/çıkış durumu	AuthProvider → Tüm uygulama
onSnapshot(users/{uid})	Profil değişikliği	AuthProvider, Sidebar, ChatArea
onSnapshot(chats)	Sohbet listesi güncellemesi	Sidebar
onSnapshot(chats/{id}/messages)	Yeni mesaj / güncelleme	ChatArea
onSnapshot(calls)	Çağrı durumu değişikliği	CallProvider, CallOverlay
onSnapshot(friendRequests)	Arkadaşlık isteği	Sidebar, FriendRequestsModal
onSnapshot(broadcastMessages)	Yeni broadcast mesajı	BroadcastChannel

Veri Tutarlılığı: Firestore'un gerçek zamanlı motoru sayesinde tüm istemciler anlık olarak senkronize olur. Mesaj durumları (sending → sent → delivered → read) otomatik olarak güncellenir. Çevrimdışı durumda Firestore SDK mesajları kuyruklar ve bağlantı yeniden kurulduğunda senkronize eder.

4. Kimlik Doğrulama (Authentication)

4.1 Google Sign-In Akışı

1. Kullanıcı "Google ile Giriş Yap" butonuna tıklar
2. Firebase Auth popup açar, Google hesabı seçilir
3. Başarılı girişte `onAuthStateChanged` tetiklenir
4. Firestore'da `users/{uid}` dokümanı oluşturulur/merge edilir
5. Profil bilgileri canlı olarak izlenmeye başlanır (`onSnapshot`)
6. Oturum `browserLocalPersistence` ile kalıcı hale getirilir

4.2 Hata Yönetimi

HATA KODU	KULLANICIYA GÖSTERİLEN MESAJ
auth/unauthorized-domain	Domain adını Firebase Auth ayarlarına ekleme talimatı
auth/popup-blocked	Popup engelleyicisini kapatma uyarısı
auth/popup-closed-by-user	Sessizce ignore edilir (hata gösterilmez)
Diğer hatalar	Hata mesajı doğrudan gösterilir

4.3 Admin Girişi

Login ekranındaki "Yönetim" bağlantısı ile açılan şifre modalı. SHA-256 hash ile doğrulama yapılır:

```
// Admin şifre hash'i (SHA-256): 44837b861990fc1aa20f725d3d4a38f34d30fd199268f5fe9d6dace38f04288d //  
Doğrulama sonrası Firestore'a kayıt: await setDoc(doc(db, 'adminUsers', user!.uid), { addedAt: new  
Date().toISOString(), email: user!.email });
```

5. Kullanıcı Profili

Her kullanıcı için Firestore'da `users/{uid}` dokümanı tutulur. Profil bilgileri `onSnapshot` ile canlı olarak güncellenir.

ALAN	TİP	ZORUNLU	AÇIKLAMA
uid	string	EVET	Benzersiz kullanıcı ID
displayName	string	EVET	Görünen ad
email	string	EVET	E-posta adresi
photoURL	string	Hayır	Profil fotoğrafı (DiceBear avatar varsayılan)
nickname	string	Hayır	Takma ad
about	string	Hayır	"Hakkında" metni
country	string	Hayır	Ülke kodu (TUR, USA, DEU...)
uin	string	Hayır	Kullanıcı kimlik numarası
onlineStatus	enum	Hayır	online away busy
role	enum	Hayır	user admin
bannedUntil	Timestamp	Hayır	Yasak bitiş zamanı
banReason	string	Hayır	Yasak sebebi
profileCompleted	boolean	Hayır	İlk kurulum tamamlandı mı?
birthDate / phone / location	string	Hayır	Kişisel bilgiler
showBirthDate / showPhone / showLocation	boolean	Hayır	Gizlilik tercihleri
broadcastHistory	string[]	Hayır	Broadcast gönderim zaman damgaları

5.1 İlk Profil Kurulumu

`profileCompleted === false` olan kullanıcılara **ProfileSetup** modalı açılır. Adım adım:

- Adım 1:** Görünen ad, takma ad, "hakkında" metni girilir
- Adım 2:** Ülke seçimi ve UIN (benzersiz kullanıcı numarası) belirlenir
- Adım 3:** Profil fotoğrafı onaylanır
- Kayıt sonrası `profileCompleted: true` ve `onlineStatus: 'online'` atanır

5.2 Profil Düzenleme

ProfileModal üzerinden yapılır (avatar veya isme tıklayarak açılır):

- **Fotoğraf:** Kamera çekimi (getUserMedia) veya dosya yükleme
- **İsim & Takma Ad & "Hakkında"** metin alanları
- **Online Durumu:** Çevrimiçi / Uzakta / Meşgul
- **Kişisel Bilgiler:** Doğum tarihi, telefon, konum
- **Gizlilik:** Doğum tarihi/telefon/konum gösterimi aç/kapa

6. Mesajlaşma Sistemi (Chat System)

6.1 Sohbet Türleri



Özel Sohbet (Private)

İki kullanıcı arasında birebir mesajlaşma. Katılımcı sayısı tam 2'dir.



Grup Sohbeti (Group)

Çok kullanıcı grupları. İsteğe bağlı şifre koruması, ülke etiketi, yönetici yönetimi.

6.2 Mesaj Türleri

TÜR	AÇIKLAMA	LİMİT	ÖZEL ÖZELLİKLER
text	Düz metin mesajı	—	Emoji, uzun metin, şifreleme
image	Base64 kodlu resim	2 MB	Şifreleme, full-screen görüntüleme
video	Base64 kodlu video	1.5 MB	Şifreleme, indirme
audio	Ses kaydı (WebM/Opus)	30sn / 900KB	Oynatıcı, şifreleme
call	Arama kaydı	—	Süre, durum (cevapsız/tamamlandı)

6.3 Mesaj Özellikleri



Mesaj Durumları

sending → sent → delivered → read. Her mesajın durumu gerçek zamanlı takip edilir.



Emoji Tepkileri

8 emoji desteği: ❤️ 😂 😞 😏 😡 👍 🔥 🍷. Toggle mantığı ile ekle/çıkart.



Şifreli Mesajlar

Base64 şifreleme. Alıcı mesajı görmek için şifre girmeli (btoa karşılaştırması).



Mesaj Silme

Soft-delete: deletedBy: string[]. Sadece silen kullanıcının ekranından kaybolur. Admin onayı ile kalıcı silme.



Görsel Görüntüleyici

Full-screen modal, zoom (+ sürükleme), indirme butonu.



Sohbet İçi Arama

Mesajlar içinde metin arama. Batch mode ile toplu mesaj seçimi.

6.4 Ses Kaydı Akışı

1. getUserMedia({ audio: true }) ile mikrofon erişimi

2. MediaRecorder ile audio/webm; codecs=opus formatında kayıt
3. Maksimum 30 saniye (otomatik durdurma)
4. Base64 dönüşümü (maks. 900KB)
5. Opsiyonel şifreleme ile Firestore'a gönderme

6.5 Video Kaydı Akışı

1. Özel dialog modalında kamera önizleme (320×240, 10fps)
2. Ön/arka kamera değiştirme desteği
3. Maksimum 10 saniye kayıt (video + ses)
4. Ön izleme sonrası gönderme veya iptal

7. Sesli & Görüntülü Arama (WebRTC Calls)

7.1 Mimarisi

CallProvider (Global Context)

- └─ activeCall: Call | null // Aktif görüşme
- └─ incomingCall: Call | null // Gelen çağrı
- └─ callError: string | null // Hata durumu

CallOverlay (UI Katmanı)

- └─ Local MediaStream (getUserMedia)
- └─ Remote MediaStream'ler (RTCPeerConnection)
- └─ RTCPeerConnection havuzu (pcs Map)
- └─ Firestore sinyalleşme (calls/{id}/signals)
- └─ Zil sesi (Web Audio API - OscillatorNode)
- └─ Mic mute / Video off kontrolleri
- └─ Çağrı süresi takibi

7.2 WebRTC Yapılandırması

```
const configuration: RTCConfiguration = { iceServers: [ { urls: 'stun:stun1.l.google.com:19302' }, {  
  urls: 'stun:stun2.l.google.com:19302' }, { urls: 'stun:stun3.l.google.com:19302' }, { urls:  
  'turn:free.turnservers.com:3478' }, ], iceCandidatePoolSize: 1, };
```

7.3 Sinyalleşme Protokolü

SİNYAL TÜRÜ	AÇIKLAMA	AŞAMA
offer	SDP Offer (çağrı başlatan taraf)	1. Çağrı başlatma
answer	SDP Answer (cevaplayan taraf)	2. Çağrı kabulü
candidate	ICE adayı (ağ keşfi)	3. Bağlantı kurulumu

Tüm sinyalleşme Firestore üzerinden calls/{callId}/signals/{signalId} koleksiyonunda gerçekleşir.

7.4 Arama Durum Makinesi

// Private arama:

Başlat → **calling** → Kabul → **ongoing** → Sonlandır → **ended**
→ Reddet / Zaman Aşımı → **ended**

// Grup arama:

Başlat → **ongoing** (beklemeden) → Katılımcı ekle/çıkar → Sonlandır

7.5 Kullanıcı Deneyimi Özellikleri

- **Zil Sesi:** Web Audio API ile sentetik ringtone (440Hz → 660Hz → 880Hz, 1.5sn aralıklarla)
- **Mikrofon Susturma:** Tek tuşla mute/unmute
- **Kamera Kapatma:** Video akışını durdurma
- **Grup Aramaya Davet:** Görüşme devam ederken yeni katılımcı ekleme
- **Görüşme Süresi:** Ongoing durumunda otomatik zamanlayıcı
- **Arama Geçmişi:** Sohbetinde call tipinde mesaj (süre, durum bilgisi)

8. Broadcast Sistemi

Küresel, ülke bazlı mesajlaşma kanalı. Kullanıcılar belirli bir ülkeye yönelik duyuru mesajları gönderebilir.

8.1 Kısıtlama Mekanizması

KURAL	DEĞER	AÇIKLAMA
Günlük limit	2 mesaj / kullanıcı	Yeni gün sıfırlanır (broadcastHistory UTC tabanlı)
Gönderim aralığı	6 saat cooldown	Son gönderimden itibaren 6 saat beklenmeli
Hedef ülke	Zorunlu	Gönderim için ülke seçimi yapılmalı

8.2 Broadcast Kanalları

BroadcastChannel bileşeni, tüm broadcast mesajlarını ülke filtresi ile görüntüler. Kullanıcının profiline kayıtlı ülke varsayılan filtre olarak atanır. Okunmamış broadcast bildirimi sidebar'da gösterilir.

8.3 Gönderim Akışı

- Kullanıcı BroadcastModal'ı açar
- Hedef ülkeyi seçer
- Kalan gönderim hakkı ve cooldown süresi kontrol edilir
- Mesaj yazılır ve gönderilir
- broadcastHistory'e zaman damgası eklenir
- Mesaj broadcastMessages koleksiyonuna kaydedilir

8.4 Desteklenen Ülkeler (60+)

Türkiye, ABD, İngiltere, Almanya, Fransa, İtalya, İspanya, Rusya, Çin, Japonya, Güney Kore, Hindistan, Brezilya, Kanada, Avustralya, Hollanda, İsveç, Norveç, Danimarka, Finlandiya, İsviçre, Avusturya, Polonya, Ukrayna, Yunanistan, Mısır, Güney Afrika, BAE, Suudi Arabistan, Meksika, Arjantin, İran, Endonezya, Malezya, Singapur, Filipinler, Vietnam, Tayland, Portekiz, Romanya, Bulgaristan, Sırbistan, Hırvatistan, Bosna Hersek, Arnavutluk, Gürcistan, Azerbaycan, Kazakistan, İsrail, Fas, Pakistan, Bangladeş, Nijerya, Kenya, Kolombiya, Şili, Peru, Küba, İrlanda, Yeni Zelanda, Macaristan, Çek Cumhuriyeti, Slovakya, Slovenya, Litvanya, Letonya, Estonya, Belarus, Moldova, Kuzey Makedonya, Karadağ.

9. Grup Sohbet Sistemi

9.1 Grup Oluřturma

1. "Yeni Grup Sohbeti" butonuna tıkanır
2. Katılımcılar seçilir (en az 1 kiři)
3. Grup adı girilir
4. Opsiyonel: grup ülkesi ve řifre belirlenir
5. Grup oluřturulur, oluřturan kullanıcı admin olur

9.2 Grup Bulma ve Katılma



Açık Gruplar

Şifresiz gruplar. Katılmak için yöneticiye istek gönderilir (groupJoinRequests).



Şifreli Gruplar

Şifre girilerek doğrudan katılım. Şifre eşleşirse kullanıcı participants dizisine eklenir.

9.3 Grup Özellikleri

- **Grup Yöneticisi:** groupMetadata.adminId ile belirlenir, ilk oluřturan admin olur
- **Ülke Etiketi:** groupCountry alanı, ülke filtresi ile grup arama
- **Yasaklı Kullanıcılar:** groupMetadata.bannedUsers dizisi
- **Grup Fotoğrafl:** DiceBear initial SVG (grup adına göre)
- **Kullanıcı Susturma:** Sohbet içinde belirli kullanıcıların mesajlarını gizleme

9.4 Grup Bilgi Görüntüleme

Sidebar'da veya sohbet başlığında açılan grup bilgi paneli: üye listesi, yönetici, ülke, řifre durumu.

10. Arkadaşlık Sistemi

10.1 Akış

// İstek Gönderme

Kullanıcı → Kullanıcı Listesi → "İstek Gönder" → friendRequests koleksiyonu

↳ status: **PENDING**

// Onaylama

Karşı Kullanıcı → FriendRequestsModal → "Onayla" → status: **APPROVED**

↳ Mevcut sohbet açılır veya yeni sohbet

oluşturulur

10.2 Durumlar

DURUM	AÇIKLAMA	UI GÖSTERGESİ
none	Arkadaşlık ilişkisi yok	"✚ İstek Gönder" butonu
pending_sent	İstek gönderildi, bekleniyor	"⌚ Bekliyor" (devre dışı)
pending_received	Gelen istek var	"📁 İstek Var" (devre dışı)
approved	Arkadaşlık onaylandı	"💬 Sohbet" butonu

10.3 FriendRequestsModal

Sidebar'daki rozet sayısı ile erişilir. Gelen istekler onaylanabilir/reddedilebilir. Gönderilen istekler listelenir. Onaylanan istekte otomatik sohbet açılır veya mevcut sohbet bulunur.

11. Yönetici Paneli (Admin Panel)

11.1 Giriş ve Yetkilendirme

SHA-256 hash ile doğrulanan şifre. Başarılı girişte adminUsers/{uid} dokümanı oluşturulur. Firestore kuralları isAdmin() fonksiyonu ile admin yetkisini tanır.

11.2 Panel Sekmeleri

SEKME	İÇERİK	İŞLEMLER
KULLANICILAR	Tüm kayıtlı kullanıcılar	Arama (isim/UIN/e-posta), ban/unban, tüm verileri silme, mesajları görüntüleme, kalıcı mesaj silme
ADMIN MESAJLARI	Kullanıcılardan admin'e gönderilen mesajlar	Listeleme, okuma
SİLİNEN MESAJLAR	Soft-delete edilmiş tüm mesajlar	İnceleme, kalıcı silme
ŞİFRELE MESAJLAR	Şifrelenerek gönderilmiş mesajlar	Listeleme, denetleme
SİLME İSTEKLERİ	Kullanıcıların kalıcı silme talepleri	Onaylama, reddetme

11.3 Yönetici Yetkileri

- Kullanıcı Banlama:** Dakika/saat/gün bazlı süreli yasaklama. bannedUntil: Timestamp olarak kaydedilir
- Kullanıcı Silme:** Tüm sohbetleri, mesajları ve profil verileri ile birlikte kalıcı silme
- Mesaj Kalıcı Silme:** deleteDoc ile Firestore'dan tamamen kaldırma
- Admin Mesajları:** toAdmin: true işaretli mesajları görüntüleme
- Etik Kuralları:** 7 varsayılan kural, ekleme/silme/düzenleme/taşıma

11.4 Etik Kuralları

#	KURAL ADI	AÇIKLAMA
1	Küfür/Hakaret	Argo ve hakaret içeren ifadeler
2	Şiddet	Şiddet içeren söylemler
3	Spam	Tekrarlayan/istenmeyen mesajlar
4	NSFW	İş yerinde uygun olmayan içerik
5	Asimov'un 1. Yasası	Bir robot insana zarar veremez veya pasif kalmak suretiyle insanın zarar görmesine göz yumamaz

6	Asimov'un 2. Yasası	Bir robot, insanlar tarafından verilen emirlere, bu emirler 1. Yasa ile çelişmediği sürece itaat etmelidir
7	Asimov'un 3. Yasası	Bir robot, 1. ve 2. Yasa ile çelişmediği sürece kendi varlığını korumalıdır

12. Yasaklama Sistemi (Ban System)

12.1 Çalışma Prensibi

1. Admin panelinde kullanıcı seçilir, ban süresi belirlenir
2. `users/{uid}/bannedUntil` alanı Timestamp olarak güncellenir
3. İstemci tarafında her 30 saniyede bir Firestore kontrolü yapılır
4. Ban süresi dolduğunda kullanıcı otomatik olarak eski haline döner

12.2 Yasaklı Kullanıcı Arayüzü

- Full-screen kırmızı tema
- Ban ikonu (Ban from Lucide)
- "Hesabınız Banlanmıştır" başlığı
- Kalan süre geri sayımı: `ss:dd:ss` (monospace font)
- Ban sebebi gösterimi (varsa)
- Tüm uygulama erişimi engellenir

12.3 Ban Süreleri

BİRİM	HESAPLAMA	KULLANIM
Dakika	$value \times 60 \times 1000$	Kısa süreli cezalar
Saat	$value \times 3600 \times 1000$	Orta süreli cezalar
Gün	$value \times 86400 \times 1000$	Uzun süreli cezalar

13. Güvenlik (Firestore Security Rules)

13.1 Yetkilendirme Fonksiyonları

```
function isAdmin() { return isSignedIn() &&
exists(/databases/$(database)/documents/adminUsers/$(request.auth.uid)); } function isSignedIn() {
return request.auth != null; } function isOwner(userId) { return isSignedIn() && request.auth.uid ==
userId; } function isParticipant(chatId) { return isSignedIn() && request.auth.uid in
get(/databases/$(database)/documents/chats/$(chatId)).data.participants; } function isGroupAdmin(chatId)
{ return isSignedIn() && request.auth.uid ==
get(/databases/$(database)/documents/chats/$(chatId)).data.groupMetadata.adminId; }
```

13.2 Validasyon Fonksiyonları

FONKSİYON	ZORUNLU ALANLAR	KISITLAMALAR
isValidUser	uid, displayName	displayName.size() > 0
isValidChat	participants, type, updatedAt	participants.size() ≥ 2, type in ['private', 'group']
isValidMessage	senderId, type, timestamp	type in ['text', 'image', 'video', 'audio', 'call']
isValidCall	participants, chatId, callerId, type, status, createdAt	participants.size() ≥ 1, geçerli type/status
isValidSignal	from, to, type, data, createdAt	type in ['offer', 'answer', 'candidate']

13.3 Katmanlı Erişim Kontrolü

KOLEKSİYON	OKUMA	YAZMA	SİLME	GÜNCELLEME KISITI
users/{uid}	Herkes (giriş yapmış)	Sadece sahibi	Admin	—
chats/{id}	Katılımcılar	Katılımcılar	Admin	—
chats/{id}/messages/{mid}	Katılımcılar	Katılımcılar	Admin	Sadece status, reactions, deletedBy
calls/{id}	Katılımcılar	Katılımcılar	Admin	—
{document=**}	Admin tam yetki (wildcard)			

13.4 12 Güvenlik Senaryosu

- Identity Spoofing:** Başkasının senderId'si ile mesaj gönderme → Firestore kuralları ile engellenir

2. **Unauthorized Access:** Katılımcı olunmayan sohbede mesaj okuma → `isParticipant()` kontrolü
3. **Shadow Update:** Profile `isAdmin` alanı ekleme → Rol `adminUsers` koleksiyonundan alınır
4. **Orphaned Message:** Var olmayan sohbede mesaj oluşturma → Chat ID validasyonu
5. **PII Leak:** Özel bilgileri toplu çekme → Firestore sorgu limitleri
6. **Malicious ID:** 1MB mesaj ID'si → Firestore döküman boyut limiti (1MB)
7. **Bypassing Invariants:** 1 katılımcılı "private" sohbet → `participants.size() ≥ 2`
8. **Impersonation:** Başkasının `displayName`'ini değiştirme → `isOwner()` kontrolü
9. **Spamming:** 1MB mesaj gövdesi → Firestore döküman boyut limiti
10. **State Corruption:** Geçmiş tarihli `updatedAt` → İstemci kontrollü, `serverTimestamp` önerilir
11. **Query Scraping:** Tüm sohbetleri toplu çekme → Katılımcı filtresi zorunlu
12. **Double Delete:** Sahip olunmayan sohbeti silme → Admin yetkisi gerekli

14. Veritabanı Şeması (Firestore Collections)

14.1 Koleksiyon Yapısı

```

users/{uid}  |— uid, displayName, email, photoURL, nickname  |— about, country, uin,
onlineStatus, role  |— bannedUntil, banReason  |— birthDate, phone, location,
showBirthDate, showPhone, showLocation  |— profileCompleted, lastSeen  |— broadcastHistory:
string[] chats/{chatId}  |— participants: string[], type: 'private'|'group'  |—
groupMetadata: { name, photoURL, createdBy, adminId, password?, bannedUsers? }  |—
lastMessage, updatedAt  |— muted?, unreadCount?, groupCountry?  |— heldBy?, holdExpiresAt?,
heldMembers?  |  |— messages/{messageId}  |— text?, senderId, timestamp  |— type,
imageUrl?, videoUrl?, audioUrl?  |— status, reactions?, deletedBy?  |— encrypted?,
imagePassword?  |— callDuration?, callStatus? calls/{callId}  |— participants,
activeParticipants  |— chatId, callerId, type, mediaType, status, createdAt  |  |—
signals/{signalId}  |— from, to, type (offer|answer|candidate)  |— data, createdAt
friendRequests/{requestId}  |— from, to, fromName, toName, fromPhoto, toPhoto  |— status,
timestamp broadcastMessages/{messageId}  |— text, senderId, senderName, senderPhoto  |—
country, createdAt, timestamp adminUsers/{uid}  // Admin yetkilendirme adminMessages/{id}
// Kullanıcı → Admin mesajları adminDeleteRequests/{id}  // Kalıcı silme talepleri
groupJoinRequests/{id}  // Grup katılım istekleri

```

15. CI/CD ve Dağıtım

15.1 GitHub Actions Pipeline

AŞAMA	İŞLEM
Tetikleyici	main branch'ine push
Secret Enjeksiyonu	VITE_GEMINI_API_KEY GitHub Secret'tan build ortamına aktarılır
Build	npm install → npm run build (Vite)
Deploy	GitHub Pages'e otomatik deploy
Sonuç	https://aydannadya31.github.io/nexus-messenger/

15.2 Firebase Deploy

```
# Firestore kuralları named database'e deploy: firebase deploy --only firestore:rules:ai-studio-6f70c272-6822-4c0b-a15d-b77d18f46fb0 # Firestore indeks deploy: firebase deploy --only firestore:indexes
```

15.3 Yerel Geliştirme

```
npm install npm run dev # → localhost:3000 (host: 0.0.0.0) npm run build # → dist/ klasörü npm run preview # → Build ön izleme npm run lint # → tsc --noEmit (tip kontrolü)
```

15.4 Firebase Auth - Yetkili Domainler

Firebase Console > Authentication > Settings > Authorized domains:

- localhost
- aydannadya31.github.io
- Her yeni domain eklenmeli (yoksa auth/unauthorized-domain hatası)

15.5 Capacitor (Android)

```
// capacitor.config.ts { appId: 'com.nexus.messenger', appName: 'NexusMessenger', webDir: 'dist' }
```

Android native paketleme için Capacitor 8 kullanılır. WebView içinde React SPA çalışır.

16. Performans ve Sınırlamalar

ÖZELLİK	SINIR	KONTROL MEKANİZMASI
Resim boyutu	Maks. 2 MB (base64)	Client-side <code>file.size</code> kontrolü
Video boyutu	Maks. 1.5 MB (base64)	Client-side kontrol
Ses kaydı süresi	Maks. 30 saniye	Otomatik durdurma (timer)
Ses base64 boyutu	Maks. 900 KB	Blob → base64 dönüşüm sonrası kontrol
Video kaydı süresi	Maks. 10 saniye	Otomatik durdurma (timer)
Günlük broadcast	Maks. 2 / kullanıcı	<code>broadcastHistory</code> sayımı
Broadcast cooldown	6 saat	Son gönderim zamanı kontrolü
Loading timeout	8 saniye	<code>setTimeout</code> ile zorunlu çıkış
Mesaj silme	Soft-delete (<code>deletedBy[]</code>)	Sadece silen kullanıcı için gizlenir
Firestore döküman	Maks. 1 MB	Firestore native limiti
Firestore sorgu	200 döküman limiti	Admin panelinde manuel limit

16.1 Mobil Optimizasyonlar

- **Responsive Sidebar:** Mobilde tam genişlik, masaüstünde 350px sabit genişlik
- **Geri Butonu:** `history.pushState` / `popstate` ile mobil navigasyon
- **Mobil Klavye:** `VisualViewport` API ile input alanının klavye üzerinde kalması
- **Safe Area:** `env(safe-area-inset-bottom)` ile çentikli ekran desteği
- **Dokunma:** `active:scale-[0.98]` ile dokunma geri bildirimi
- **Yakınlaştırma Engeli:** `user-scalable=no` ile input zoom sorununun önlenmesi
- **Font:** `font-size: 16px !important` ile iOS otomatik zoom engeli

17. Proje Dosya Yapısı

```
AFCB messenger/
├─ src/
│   ├─ main.tsx // Uygulama giriş noktası
│   ├─ App.tsx // Ana uygulama bileşeni
│   ├─ index.css // Tailwind CSS + @theme
│   ├─ types.ts // Tüm TypeScript tipleri
│   ├─ lib/
│   │   ├─ firebase.ts // Firebase init + Auth helpers
│   │   └─ utils.ts // cn() yardımcısı
│   └─ components/
│       ├─ AuthProvider.tsx // Auth context + Firestore profil
│       ├─ CallProvider.tsx // WebRTC context
│       ├─ Login.tsx // Giriş ekranı
│       ├─ Sidebar.tsx // Sohbet listesi (719 satır)
│       ├─ ChatArea.tsx // Mesajlaşma (1991 satır)
│       ├─ CallOverlay.tsx // Arama arayüzü (634 satır)
│       ├─ AdminPanel.tsx // Yönetim paneli (730 satır)
│       ├─ BroadcastChannel.tsx // Broadcast kanalı
│       ├─ BroadcastModal.tsx // Broadcast gönderme
│       ├─ NewChatModal.tsx // Yeni sohbet/grup (459 satır)
│       ├─ ProfileModal.tsx // Profil düzenleme (343 satır)
│       ├─ ProfileSetup.tsx // İlk profil kurulumu
│       ├─ FriendRequestsModal.tsx // Arkadaşlık istekleri
│       └─ ErrorBoundary.tsx // Hata yakalama
├─ functions/ // Firebase Cloud Functions
├─ android/ // Capacitor Android projesi
├─ .github/workflows/ // GitHub Actions CI/CD
├─ capacitor.config.ts
├─ firebase-applet-config.json
├─ firebase.json
├─ firestore.rules // Güvenlik kuralları (150 satır)
├─ firestore.indexes.json
├─ security_spec.md // Güvenlik şartnamesi
├─ CHAT_HISTORY.md // Geliştirme geçmişi
├─ README.md
├─ package.json
├─ vite.config.ts
├─ tsconfig.json
├─ index.html
├─ .env.example
├─ .firebaserc
└─ .gitignore
```

└─ metadata.json

└─ .opencode/ // AI agent yapılandırması

18. Özet ve Değerlendirme

Nexus Messenger (A+F=C.B), Firebase ekosisteminin tüm gücünü kullanan, modern ve kapsamlı bir mesajlaşma uygulamasıdır. Web ve Android platformlarında çalışan uygulama; gerçek zamanlı mesajlaşma, WebRTC ile sesli/görüntülü arama, kapsamlı yönetici paneli, broadcast sistemi, şifreli mesajlaşma ve arkadaşlık sistemi gibi özellikleri tek bir platformda birleştirir.

Öne Çıkan Güçlü Yönler

#	ALAN	AÇIKLAMA
1	Gerçek Zamanlı Mimari	Firestore onSnapshot ile anlık veri senkronizasyonu. Mesaj durumları, profil değişiklikleri, çağrı durumları anında tüm istemcilere yansır.
2	Kapsamlı Yönetim	Detaylı admin paneli ile kullanıcı, mesaj, ban ve etik kuralları yönetimi. 5 ayrı sekmede organize edilmiş tam kontrol.
3	İletişim Çeşitliliği	Metin, resim, video, ses kaydı, sesli arama, görüntülü arama — 6 farklı iletişim kanalı.
4	Güvenlik	Firestore kuralları, SHA-256 hash ile admin doğrulama, şifreli mesajlar, yasaklama sistemi, 12 güvenlik senaryosu.
5	Mobil Uyumluluk	Responsive tasarım, Capacitor Android, VisualViewport klavye yönetimi, safe area desteği.
6	Küresel Erişim	60+ ülke desteği ile broadcast sistemi, ülke bazlı filtreleme, çok dilli altyapı.
7	Modern Teknoloji	React 19, TypeScript 5.8, Tailwind CSS 4, Vite 6, Motion (Framer Motion) 12.

Mevcut Sınırlamalar ve Açık Sorunlar

#	SORUN	ETKİ	ÇÖZÜM
1	Firestore kuralları doğru database'e deploy edilmedi	Admin panelinde "Kullanıcı mesajları yüklenirken hata"	firebase deploy --only firestore:rules:...
2	VITE_GEMINI_API_KEY GitHub Secret eksik	AI özellikleri çalışmıyor	GitHub repo Secrets'a eklenecek, redeploy
3	Profil güncellemeleri sidebar'da anlık yansımıyor	Kullanıcı yeniden giriş yapmalı	onSnapshot bağlantısının iyileştirilmesi

Gelecek Geliştirme Potansiyeli

- AI Entegrasyonu:** Google Gemini API ile akıllı mesaj önerileri, moderasyon, çeviri
- Uçtan Uca Şifreleme:** Soft-delete modelinden kalıcı silme + E2EE
- Push Bildirimleri:** Firebase Cloud Messaging ile arka planda bildirim
- Medya Upload:** Base64 yerine Firebase Storage ile dosya yükleme
- iOS Desteği:** Capacitor iOS ile App Store dağıtımı

- **Sesli/Görüntülü Mesaj:** WhatsApp tarzı basılı tut-conuş özelliği
- **Durum (Story) Paylaşımı:** 24 saat silinen geçici medya paylaşımı

Nexus Messenger — A+F=C.B

Kapsamlı Sistem Sunumu • 12 Haziran 2026

Firebase Project: gen-lang-client-0308378658 • Database: ai-studio-6f70c272-6822-4c0b-a15d-b77d18f46fb0

Canlı: aydannadya31.github.io/nexus-messenger/